AutoVet Security and Authentication

Where the security routing lives in the codebase, and how login tokens work

1. Main Security Routing (Backend)

`backend/routes/api.php` is the main file. Every API URL is defined here, grouped by who is allowed to use it.

Location	Section	Who can use it
`routes/api.php:43`	**Public routes**: login, register, Google sign-in, password reset	Anyone (rate-limited)
`routes/api.php:49-50`	`/auth/google` and `/login`: where logging in happens	Anyone, max 10 tries per minute
`routes/api.php:105`	**The main security gate**: `auth:sanctum`	Must be logged in
inside that group	`role:...` sub-groups (e.g. inventory, walk-in)	Only certain roles
`routes/api.php:325`	`role:super_admin` group	Super Admin only

Supporting files

File	What it does
`backend/bootstrap/app.php:30-38`	Registers the security middleware: `auth` (logged in?), `role` (right role?), `maintenance`, and `SecurityHeadersMiddleware` on every API response.
`backend/app/Http/Middleware/RoleMiddleware.php`	The role check. Returns **403** if the user's role is not allowed. Super Admin passes every check.
`backend/app/Enums/Roles.php`	Defines the role groups used by the routes: `adminRoles()`, `dashboardRoles()`, `clinicalRoles()`.
`backend/app/Http/Middleware/SecurityHeadersMiddleware.php`	Adds browser security headers: HSTS (HTTPS only), Content-Security-Policy, X-Frame-Options, and others.

2. Token and Authentication: Laravel Sanctum

> **In simple terms:** after you log in, the server gives you a long random **token** (a "personal access token"). The app sends it with every request in a header: `Authorization: Bearer <token>`. The server checks it on each request.

Backend

Step	File
Login: check email and password with `Hash::check` (bcrypt), then create the token	`backend/app/Http/Controllers/AuthController.php:183` (`login`), token created around `:302`
Google login: verify with Google, then create a token	`backend/app/Http/Controllers/GoogleAuthController.php:75`
Token settings: **expires after 30 days**	`backend/config/sanctum.php:8`
Two account types, each with its own Sanctum guard: `admins` (staff) and `portal_users` (pet owners)	`backend/config/auth.php`

Frontend

Step	File
Admin app saves the token after login (`localStorage`)	`frontend/admin/src/context/AuthContext.jsx:42`
Admin app attaches it to every request	`frontend/admin/src/api.js:43`
Portal saves and attaches it	`frontend/portal/src/context/AuthContext.tsx:56`, `frontend/portal/src/api.ts:27-29`
Pages hidden from the wrong roles	`frontend/admin/src/components/auth/ProtectedRoute.jsx`, used in `frontend/admin/src/app/router.jsx`

> **Important:** the frontend page guard (`ProtectedRoute`) only *hides* pages. **The real security is on the backend** (`auth:sanctum` + `role:`), because anyone can call the API directly.

3. The Login Flow

Login form → `POST /api/login` → Server checks password hash → Server creates Sanctum token (30 days) →

Frontend saves it in `localStorage` → Every request sends `Bearer <token>` → `auth:sanctum` checks token, then `role:` checks role

4. Known Weakness (If the Panel Asks)

The token is stored in **`localStorage`**. This is common for this kind of app, but if an attacker ever got JavaScript running on the page (an XSS attack), they could read the token. The Content-Security-Policy header reduces that risk.

Possible future improvement: switch to Sanctum's cookie-based login (httpOnly cookies, which JavaScript cannot read).